Project II : Machine Learning, Deep Learning using backpropagation in python

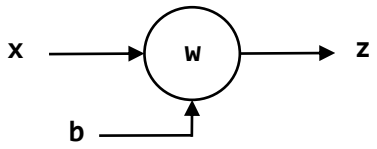
This project explores the theory and implementation of deep learning and neural networks. We start by understanding how a neural network learns using backpropagation. Then, we build a simple neural network for handwritten digit recognition without using any library. After that, we implement the same task using Pytorch.

We also experiment with decision boundaries to visualize how the model separates different classes. Finally, we create a simple web interface to test the model.

1- Theory behind deep learning, neural networks, and backpropagation.
2- Handwritten Digit recognition neural network without libraries (pytorch).
3- Handwritten Digit recognition using pytorch.
4- Experimentation with **decision boundaries.**
5- Web interface implementation.

Theory behind deep learning, neural networks, and backpropagation

1. Basic Model:
Let's define first a basic linear neuron with two parameters, a weight w and a bias b and one input x, represents a characteristic that our model would learn such that its output z is: $z = w \cdot x + b$



2. Activation functions:
In large systems, many neurons are used to model high complexity problems. The input vector of each neuron is as large as the data entries x, making computation longer. One way to optimize the computation process is to introduce a nonlinearity in the neurons output, to prioritize firing[1] neurons and killing low impact ones.

$$a = activation(z)$$

Different **activation functions** can be used depending on the application:

$$sigmoid(z) = \frac{1}{1+e^{-z}}$$

Used for binary classification and squashes the output between 0 and 1.

$$ReLu(z) = \{z > 0 : z, 0\}$$

Common for its simplicity and efficiency.

$$soft\max(z) = \frac{e^{z-z_{max}}}{\sum_{i=0}^{n} e^{z_i}}$$

SoftMax is used for multi-class probabilistic classification (output the probability that the input falls in a category or the other).

[1] Firing: significant output, non-negligible contribution, z>0

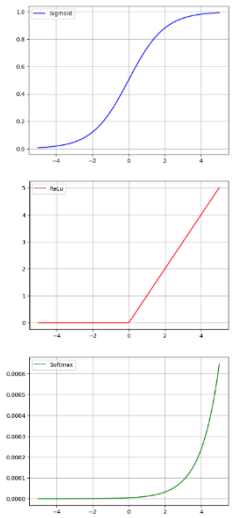
```
Import NumPy as np
import matplotlib.pyplot as plt

z = np.linspace(-5,5,100)

class Activation:
    def __init__(self,z):
        self.z = z

    def sigmoid(self):
        return 1/(1+np.exp(-self.z))
    def relu(self):
        return np.where( z>0, z,0)
    def softmax(self):
        return np.exp(self.z -  np.max(self.z))/np.sum(np.exp(self.z))

plt.plot(z, Activation(z).sigmoid(), color = "blue" ,label = "Sigmoid")
plt.plot(z,Activation(z).relu(), color = "red" , label = "ReLu")
plt.plot(z,Activation(z).softmax(), color = "green" , label = "Softmax")
plt.grid()
plt.legend()
```



3. Vectorization:

In a real example a model takes more than one input parameter x, a data set represented as a vector, so to ease computation we will use vector/Matrix representation.

So, we define a vector composed of n-characteristics modeled by a parameter x_i

$$X_n = (x_1, x_2, x_3, x_4, x_5, \dots x_n)$$

This implies that each x_i must weighted by w_i thus:

$$W_n = (w_1, w_2, w_3, w_4, w_5, \dots w_n)$$

Evidently so, the output z becomes $Z = b + \sum_{i=1}^{n} w_i x_i$

We can clearly notice that the sum term is a dot product between W_n and X_n.

$$\boxed{Z = W_n \cdot X_n + b}$$

4. Multi-neuron architecture:

Now that we have a vectorized version of our neuron, it will be easier to derive a "Layer of neurons" such that, the input X_n is passed on to multiple neurons in parallel de calculate Z_m that could represent multiple classes.

Example : the input is an image of a dog; the output is different races of dogs. If we label many images of dogs and feed them to a neural network, and say that its output vector is different dog races index form 1 to m, the higher z_i is the more the model predicts it to be the right label (race).

Image of a dog label_3

Neural network

$$Z_m = \begin{cases} 0.1 & label_1 \\ 0.02 & label_2 \\ 0.76 & label_3 \\ 0.03 & label_4 \\ 0.09 & label_5 \\ 0.01 & label_6 \end{cases}$$

- The model is 76% sure that the image is from class 3 with the label_3 (a race for example or any other feature).

Consider a unique Layer of m neuron, then we will have m outputs, $Z_m^1 = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}$

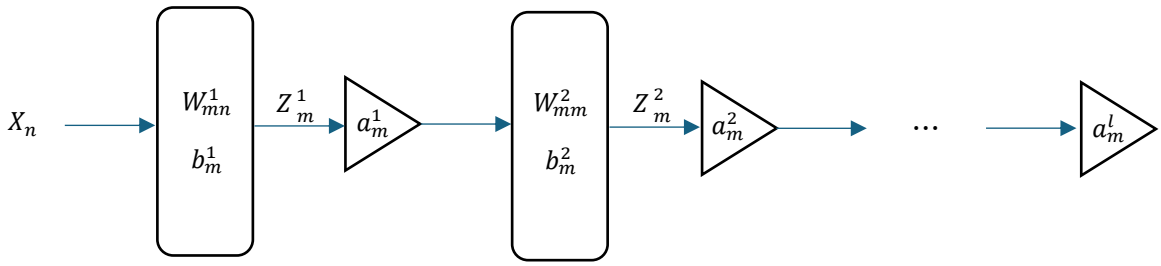
$$\begin{cases} z_1 = W_{1n} \cdot X_n + b \\ z_2 = W_{2n} \cdot X_n + b \\ \vdots \\ z_m = W_{mn} \cdot X_n + b \end{cases} \longrightarrow \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}$$

We can finally extract a matrix formula for any layer l with m neurons(rows) and n inputs(columns):

$$\boxed{Z_m^l = W_{mn}^l * X_n + b_m^l \qquad or \qquad Z_m^l = W_{mn}^l * X_n{}^T + b_m^l}$$

X_n when the input is a column vector, $X_n{}^T$ when it is a row vector.

5. Multi-layer architecture:
Example of an l layer neural network:



Where: $Z_m^2 = W_{mm}^2 * a_m^1 + b_m^2$ *And* $a_m^1 = activation(Z_m^1) = activation(W_{mn}^1 * X_n + b_m^1)$

Assuming no activation fuction, the output of each layer l is then computed like that, successive matrix multiplications and additions, therefore:

$$Z^l = W^1 W^2 W^3 * X_n + W^2 W^3 b^1 + W^3 b^2 + b^3 \cdots$$

$$\boxed{Z^l = \left(\prod_{i=1}^{l} W^i\right) * X_n + \sum_{i=1}^{l} \left[\left(\prod_{j=i+1}^{l} W^j\right) * b^i\right]}$$

6. Bash training:
Often, we need to train over large data sets with different parameters x, an example of that would be:

- A deep network to estimate house prices using a set of parameters, surface, year of construction …
- The training data could be represented as a matrix of n parameters (rows), and p examples of houses (columns):

$$X_{np} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

$$\longrightarrow \quad \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix} = \begin{pmatrix} z_{11} & z_{12} & \cdots & z_{1p} \\ z_{21} & z_{22} & \cdots & z_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ z_{m1} & z_{m2} & \cdots & z_{mp} \end{pmatrix}$$

The previous formulas derived still hold, the difference is that the output is now composed of p column vectors Z_{mp}, being the output of each column input X_{np}.

$$Z^{\,l}_{\;mp} = W^l_{mn} * X_{np} + b^l_m \qquad or \qquad Z^{\,l}_{\;mp} = W^l_{mn} * X_{np}{}^{\,T} + b^l_m$$

This optimizes the computation for the CPU, preventing training over the sample data one by one in a loop.

This structure is scalable for larger datasets, such as batches of images, where each image can be vectorized and processed in parallel.

7. Loss functions:

To train a model, it requires comparing with the targeted output and adjusts weights values to best fit the wanted target. We model the correctness of the neural network using a Loss function that describes how far off the prediction is from the target.

Let's define a target vector $\hat{Z}_m = \begin{pmatrix} \hat{z}_1 \\ \hat{z}_2 \\ \vdots \\ \hat{z}_m \end{pmatrix}$ $or\ for\ batch\ traning\ \ \hat{Z}_{mp} = \begin{pmatrix} \hat{z}_{11} & \hat{z}_{12} & \cdots & \hat{z}_{1p} \\ \hat{z}_{21} & \hat{z}_{22} & \cdots & \hat{z}_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{z}_{m1} & \hat{z}_{m2} & \cdots & \hat{z}_{mp} \end{pmatrix}$

A simple loss function would be the average of difference between the predicted values Z_m and the real values $\hat{Z}_m$.

$$L(\hat{Z}_m, Z_m) = \frac{1}{m}\sum_{i=1}^{m} \hat{Z}_i - Z_i \qquad or \qquad L(\hat{Z}_{mp}, Z_{mp}) = \frac{1}{mp}\sum_{j=1}^{p}\left(\sum_{i=1}^{m} \hat{Z}_{ij} - Z_{ij}\right)$$

Similarly for activation function, the loss function is chosen depending on the type of data, the purpose … the MSE (mean squared error) loss function is used in linear regression[2], its error is squared to penalize large deviation.

$$L(\hat{Z}_m, Z_m) = \frac{1}{m}\sum_{i=1}^{m} (\hat{z}_i - z_i)^{\,2} \qquad or \qquad L(\hat{Z}_{mp}, Z_{mp}) = \frac{1}{mp}\sum_{j=1}^{p}\left\{\sum_{i=1}^{m} (\hat{z}_i - z_i)^{\,2}\right\}$$

But for classification neural networks which is the case of this project, we use a completely different function named **cross entropy**. Its specificity is that it must be used with probabilities, so the output z_i must range between 0 and 1 $\{0,1\}$. $z_i \in \{0,1\}$ meaning that the last layer must be activated using SoftMax.

$$L(\hat{Z}_m, Z_m) = -\sum_{i=1}^{m} \hat{z}_i \log(z_i) \qquad or \qquad L(\hat{Z}_m, Z_m) = -\frac{1}{p}\sum_{j=1}^{p}\sum_{i=1}^{m} \hat{z}_{ip} \log(z_{ip})$$

[2] Practical exemple in the Annex

In batch training configuration, we must average over the number of samples thus we divide only by p, the number of samples.

Now that we defined all the mathematical tools and institutions required, and derived formulas to evaluate the model correctness, we can move on to the backpropagation algorithm.

8. backpropagation:
schematic

The backpropagation algorithm relies on one stem concept : partial derivatives of the loss function with respect to the weights and biases.

In othoers words we compute the contribution of each weight and bias to the loss function.

$$\frac{d}{dw_{ij}} L(\hat{Z}_m, Z_m) \quad and \quad \frac{d}{db_i} L(\hat{Z}_m, Z_m)$$

Deriving the loss function with respect to the weights is long and tedious. genuinely we use the chain rule:

Case with no activation function:

$$\frac{d}{dw_{ij}} L(\hat{Z}_m, Z_m) = \frac{d}{dZ_m} L(\hat{Z}_m, Z_m) * \frac{dZ_m}{w_{ij}}$$

Case with activation function:

$$a = activation(z) \longrightarrow A_m = activation(Z_m)$$

Recall: $\hat{A}_m$ is the activation of the target vector $\hat{Z}_m$:

$$\boxed{\frac{d}{dw_{ij}} L(\hat{A}_m, A_m) = \frac{d}{dA_m} L(\hat{A}_m, A_m) * \frac{dA_m}{dZ_m} * \frac{dZ_m}{w_{ij}}}$$

In general, here is the formula:

$$\boxed{\frac{dl}{dw} = \frac{dl}{da} * \frac{da}{dz} * \frac{dz}{dw} \quad and \quad \frac{dl}{db} = \frac{dl}{da} * \frac{da}{dz} * \frac{dz}{db}}$$

Now that we have the contribution of each weight and biases, we can adjust them to fit the target, but how?

Since we have the derivative $\frac{dl}{dw}$, let us go back to calculus and the meaning of a derivative.

$$\begin{cases} \frac{dl}{dw} > 0 & \textbf{case 1}\text{: the loss is increasing with respect to w.} \\ \frac{dl}{dw} = 0 & \textbf{case 2}\text{: the loss is unaffected by changes in w.} \\ \frac{dl}{dw} < 0 & \textbf{case 3}\text{: the loss is decreasing with respect to w.} \end{cases}$$

Recall: our objective is to minimize the loss, so when $\frac{dl}{dw} > 0$ w must be decreased and similarly, when $\frac{dl}{dw} < 0$ w must be increased proportionally to its own contribution and a learning rate α.

$$\boxed{w_{new} = w - \alpha * \frac{dl}{dw} \qquad or \qquad b_{new} = b - \alpha * \frac{dl}{db}}$$